RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

Moduļa “Datu bāzu tehnoloģijas”

NOSLĒGUMA DARBA SATURS

2026./2027.m.g.

SATURS

1. NOSLĒGUMA DARBA IZSTRĀDE ... 3

1.1. Noslēguma darba norise un veicamie uzdevumi ... 3

1.2. Iesniedzamie materiāli ... 3

1.3. Prasības noslēguma darba izstrādei ... 4

1.3.1. Prasības plānotajai informācijas sistēmai ... 4

1.3.2. Prasības datu bāzes sistēmas prototipam ... 4

2. PRASĪBAS NOSLĒGUMA DARBA SATURAM ... 6

2.1. Tirgus izpētes rezultāti ... 6

2.2. Projekta dokumentācija ... 6

2.2.1. Titullapa ... *7*

2.2.2. Saturs ... *7*

2.2.3. Ievads ... *7*

2.2.4. Uzdevuma nostādne ... *7*

2.2.5. Sistēmas koncepcijas izvērtējums ... *11*

2.2.6. Datu bāzes sistēmas projektēšana ... *12*

2.2.7. Datu bāzes sistēmas realizācija un testēšana ... *18*

2.2.8. Nobeigums ... *20*

2.2.9. Informācijas avoti ... *21*

2.2.10. Pielikumi ... *21*

2.3. Prezentācija sistēmas koncepcijas prezentēšanai ... 21

2.4. Pašvērtējums ... 21

3. NOSLĒGUMA DARBA AIZSTĀVĒŠANA ... 22

PIELIKUMI ... 24

1.pielikums. Titullapas paraugs ... 24

2.pielikums. Satura rādītājs ... 25

3.pielikums. Sistēmas lietotāju apraksta piemērs ... 26

4.pielikums. Objektorientētā konceptuālā datu modeļa apraksta piemērs ... 27

5.pielikums. Klašu diagrammas piemērs ... 28

6.pielikums. ER diagrammas piemērs ... 29

7.pielikums. Datu glabāšanas struktūras apraksta piemērs ... 30

8.pielikums. Datu bāzes tabulu saišu shēmas piemērs ... 31

9.pielikums. Datu bāzes realizācijas apraksta piemērs ... 32

10.pielikums. Funkcionālo prasību realizācijas un testēšanas piemērs ... 33

2. PRASĪBAS NOSLĒGUMA DARBA SATURAM

2.1. Tirgus izpētes rezultāti

Tirgus izpētes mērķis ir identificēt risināmo problēmu, pamatot izstrādājamās datu bāzes sistēmas nepieciešamību un iegūt informāciju, kas kalpo par pamatu sistēmas koncepcijas izstrādei.

Tirgus izpētes rezultāti jāiesniedz atsevišķa dokumenta veidā, kas sagatavots atbilstoši metodiskajiem norādījumiem un mācību darbu noformēšanas prasībām.

Dokumentā jāidentificē problēma, kuru paredzēts risināt, izstrādājot datu bāzes sistēmu, un jāpamato tās aktualitāte. Jāveic tirgū pieejamo līdzīgo risinājumu izpēte, analizējot vismaz divas līdzīgas informācijas sistēmas. Analīzē jāizvērtē to galvenā funkcionalitāte, priekšrocības un trūkumi, kā arī jānosaka, kādas iespējas būtu lietderīgi izmantot vai pilnveidot savā sistēmā. Analīzes rezultātus ieteicams apkopot salīdzinājuma tabulā.

Pamatojoties uz veikto tirgus izpēti, jāformulē secinājumi, kuros pamatota izstrādājamās sistēmas ideja, noteiktas tās galvenās priekšrocības salīdzinājumā ar analizētajiem risinājumiem un iezīmēts sistēmas turpmākās attīstības virziens. Tirgus izpētes rezultātiem jākļūst par pamatu sistēmas koncepcijas izstrādei.

Dokumentā sniegtajai informācijai jābūt pamatotai ar pierādījumiem. Ieteicams iekļaut analizēto sistēmu ekrānattēlus (screenshots), salīdzinājuma tabulas, attēlus un atsauces uz izmantotajiem informācijas avotiem.

Tirgus izpētes rezultātā izglītojamajam jāspēj pamatot savas sistēmas ideju, noteikt tās galveno funkcionalitāti, identificēt priekšrocības salīdzinājumā ar esošajiem risinājumiem un definēt sistēmas turpmākās attīstības virzienu.

2.2. Projekta dokumentācija

Projekta dokumentācija (*paskaidrojošais raksts*) sagatavojama atbilstoši metodiskajiem norādījumiem un mācību darbu noformēšanas prasībām.

Dokumentācijā secīgi jāatspoguļo datu bāzes sistēmas izstrādes process – sākot no sistēmas koncepcijas izstrādes un tās izvērtēšanas līdz datu bāzes projektēšanai, sistēmas funkcionālo prasību realizēšanai un testēšanai.

Obligātās projekta dokumentācijas sastāvdaļas ir: Titullapa, Saturs, Ievads, Uzdevuma nostādne, Sistēmas koncepcijas izvērtējums (SVID analīze), Datu bāzes sistēmas projektēšana, Datu bāzes sistēmas realizācija un testēšana, Nobeigums, Informācijas avoti un Pielikumi.

Katras sadaļas saturs un noformēšanas prasības ir aprakstītas turpmākajās metodisko norādījumu nodaļās.

2.2.1. Titullapa

Noslēguma darba titullapu noformē atbilstoši paraugam (skat. 1.pielikumu). Titullapu nevajag ierāmēt!

2.2.2. Saturs

Saturā uzrāda daļu, nodaļu un apakšnodaļu, pielikumu numerāciju un nosaukumus, kā arī visu sadaļu nosaukumus. Saturā iekļauj visus teksta virsrakstus līdz 3. līmenim ieskaitot. Saturam ir jāiekļaujas vienā lapā, kuru nenumurē. Satura rādītāja paraugs ir dots 2.pielikumā.

> Turpmāk minētie nosaukumi veido obligāto projekta dokumentācijas nodaļu un apakšnodaļu struktūru. Dokumentācijā tie jānumurē atbilstoši satura rādītāja paraugam.

2.2.3. Ievads

Ievadā īsi raksturo projekta priekšmeta jomu un problēmu, kuru plānots risināt, izstrādājot datu bāzes sistēmu. Jāapraksta aktuālā situācija izvēlētajā jomā, norādot, ar kādām grūtībām vai nepilnībām saskaras potenciālie lietotāji un kāpēc ir nepieciešams izstrādāt jaunu vai pilnveidotu risinājumu.

Pamatojoties uz veikto tirgus izpēti, īsi jāraksturo esošā situācija tirgū un jānorāda, kādi līdzīgi risinājumi ir analizēti. Nav nepieciešams atkārtoti aprakstīt tirgus izpētes rezultātus vai detalizēti salīdzināt analizētās sistēmas, bet jāsniedz kopsavilkums par galvenajiem secinājumiem, kas pamato izvēlētās sistēmas izstrādi.

Ievadā jāraksturo arī sistēmas mērķauditorija, norādot, kam sistēma ir paredzēta, kādas ir šo lietotāju vajadzības un kādus ieguvumus viņi iegūs, izmantojot izstrādājamo sistēmu. Noslēgumā īsi jānorāda, kāpēc izvēlētais risinājums ir aktuāls un lietderīgs, kā arī jāiezīmē sistēmas galvenā ideja un tās paredzamās priekšrocības salīdzinājumā ar esošajiem risinājumiem.

Ievads ir pirmā lapa, kas tiek numurēta ar skaitli “3”.

2.2.4. Uzdevuma nostādne

Šajā sadaļā tiek aprakstīta izstrādājamās datu bāzes sistēmas koncepcija un definētas tās galvenās īpašības, funkcionalitāte un lietotāju mijiedarbība ar sistēmu. Sadaļa sastāv no ievadteksta un vairākām apakšsadaļām.

1. Ievadteksts

Ievadteksta pirmajā teikumā skaidri jāformulē noslēguma darba uzdevums. Piemēram: "*Noslēguma darba uzdevums ir izveidot datu bāzes sistēmas prototipu tiešsaistes platformai, kurā lietotāji var lasīt klasiskās un oriģinālās grāmatas, publicēt savus darbus un mijiedarboties ar citiem platformas lietotājiem*."

Turpmākajā ievadtekstā jāpamato sistēmas izstrādes nepieciešamība, īsi raksturojot galvenās problēmas, kuras plānots risināt, sistēmas priekšrocības un galvenās lietotāju vajadzības. Ievadtekstam jābalstās uz tirgus izpētes rezultātiem, taču tajā nav atkārtoti jāapraksta tirgus izpētes saturs vai analizētie risinājumi.

Ievadtekstā var minēt arī sistēmas iespējamās nākotnes funkcijas un attīstības perspektīvas, taču **turpmākajās apakšsadaļās jāapraksta tikai tā funkcionalitāte, kuru paredzēts projektēt vai realizēt noslēguma darba ietvaros**.

Uzdevuma nostādne ievadteksta piemērs ir dots 2.attēlā

1. UZDEVUMA NOSTĀDNE

Noslēguma darba uzdevums ir izstrādāt datu bāzes sistēmas prototipu tiešsaistes platformai, kurā lietotāji var lasīt klasiskās un tradicionālās grāmatas, kā arī publicēt savus oriģināldarbus plašākai auditorijai.

Šādas sistēmas izveide ir aktuāla, jo pašlaik ir maz platformu, kas vienuviet nodrošina piekļuvi gan klasiskajai literatūrai, gan lietotāju radītajiem darbiem. Izstrādājamā sistēma veicinās literatūras pieejamību, atbalstīs jauno autoru darbu publicēšanu un palīdzēs lietotājiem ērti atrast sev interesējošo saturu. Sistēma būs paredzēta lasītājiem, autoriem un administratoriem, nodrošinot katrai lietotāju grupai tās vajadzībām atbilstošu funkcionalitāti.

2.att. Uzdevuma nostādnes ievadteksta piemērs

2. Sistēmas funkcionalitāte

Šajā apakšsadaļā kopsavilkuma veidā jāapraksta sistēmas funkcionalitāte, kas tiks iekļauta sistēmas koncepcijā un realizēta un/vai projektēta noslēguma darba ietvaros.

- ✓ ***Vispārējā sistēmas funkcionalitāte***

  Jāapraksta galvenās sistēmas iespējas, piemēram, lietotāju reģistrācija un autentifikācija, profila pārvaldība, informācijas publicēšana, citas sistēmas funkcijas.

  Nav nepieciešams detalizēti aprakstīt katras funkcijas darbības algoritmu — pietiek raksturot sistēmas galvenās iespējas.

- ✓ ***Datu pārvaldības funkcionalitāte***

Šajā apakšsadaļā kopsavilkuma veidā jānorāda datu bāzes plānotais saturs, raksturojot, kāda informācija tiks glabāta datu bāzē (piemēram: "Datu bāzē tiks glabāta informācija par lietotājiem, grāmatām, kā arī lietotāju jautājumiem administrācijai.")

> Nedrīkst aprakstīt datu bāzes tabulas, atribūtus, primārās vai ārējās atslēgas, datu tipus vai citus fiziskā datu modeļa elementus, jo tie tiek izstrādāti vēlāk datu bāzes projektēšanas posmā.

Šajā apakšsadaļā arī jāapraksta sistēmā paredzētās datu pievienošanas, labošanas un dzēšanas iespējas, norādot, kādus datus un kādas lietotāju lomas drīkst pārvaldīt. Aprakstā jāiekļauj tikai noslēguma darba ietvaros projektējamās un realizējamās datu pārvaldības darbības.

- ✓ ***Funkcionalitāte datu izgūšanai un apstrādei***

Jāapraksta visas sistēmā paredzētās funkcionālās prasības datu izgūšanai un apstrādei, piemēram, datu meklēšana, filtrēšana, kārtošana, vairāku savstarpēji saistītu datu kopu apvienošana, kopvērtību aprēķināšana un grupēšana.

3. Sistēmas lietotāji un mijiedarbība ar sistēmu

Šajā sadaļā jāapraksta sistēmas lietotāji, to lomas un mijiedarbība ar sistēmu. Sadaļa sastāv no divām apakšsadaļām: **Sistēmas lietotāji** un **Lietotāju uzvedība un mijiedarbība ar sistēmu**.

- ✓ ***Sistēmas lietotāji***

Šajā apakšsadaļā jāapraksta visas sistēmas lietotāju lomas, to galvenās funkcijas, piekļuves tiesības un atbildība sistēmā. Obligāti jāparedz vismaz šādas lietotāju lomas: viesis, reģistrēts lietotājs un administrators. Ja nepieciešams, reģistrēto lietotāju var sadalīt vairākās apakšlomās (piemēram, lasītājs, autors, moderators u. c.), pamatojot šādu sadalījumu.

Katrai lietotāju lomai jānorāda:

- lomas mērķis un funkcija sistēmā;
- pieejamā funkcionalitāte;
- piekļuves tiesības un ierobežojumi.

Šajā apakšsadaļā jāiekļauj atsauce uz **USE CASE diagrammu**, kas attēlo sistēmas lietotāju mijiedarbību ar sistēmu. Diagrammu ieteicams ievietot pielikumā. Diagrammai jāatbilst šādām prasībām.

- Diagrammā jāattēlo visas sistēmas lietotāju lomas.
- USE CASE diagrammā jāattēlo tikai tās darbības, kuras veic lietotāji. Sistēmas iekšējās darbības (piemēram, ievades datu validācija, paroles

šifrēšana, datu saglabāšana u. c.) nav lietošanas gadījumi un diagrammā nav attēlojamas.

- Katram lietotājam jābūt piesaistītiem loģiskiem un savstarpēji atšķirīgiem lietošanas gadījumiem.
- Ja viena lietotāju loma ietver citas lietotāju lomas funkcionalitāti (piemēram, moderators var veikt visas autora darbības un papildus tam arī moderēt saturu), USE CASE diagrammā jāizmanto aktieru mantošana (generalization). Kopīgie lietošanas gadījumi jāattēlo tikai vienu reizi, izmantojot mantošanas saiti, nevis atkārtoti zīmējot identiskas asociācijas.
- Nepieciešamības gadījumā jāizmanto «**include**» un «**extend**» saites, pamatojot to izmantošanu.
- Diagramma jāizstrādā atbilstoši UML standartam, izmantojot pareizos elementus un nodrošinot pārskatāmu izkārtojumu.

Lietotāju apraksta piemērs ir pieejams 3. pielikumā.

✓ ***Lietotāju uzvedība un mijiedarbība ar sistēmu***

Šajā apakšsadaļā jāapraksta lietotāju tipiskā rīcība sistēmā, darbību secība dažādās situācijās un sistēmas reakcija uz lietotāja veiktajām darbībām.

Katrai lietotāju lomai ieteicams aprakstīt vairākus tipiskus lietošanas scenārijus. Piemēram:

> "Viesis atver sistēmu, meklē grāmatu pēc nosaukuma, apskata tās aprakstu un vēlas pievienot komentāru. Sistēma konstatē, ka lietotājs nav autentificējies, un piedāvā reģistrēties vai autorizēties."

> "Autors autorizējas sistēmā, pievieno jaunu darbu, augšupielādē vāka attēlu, saglabā darbu kā melnrakstu un vēlāk publicē to."

> "Reģistrācijas laikā viesis izvēlas darbību "Reģistrēties" un ievada pieprasītos datus, piemēram, e-pasta adresi un paroli. Pēc datu ievades sistēma automātiski pārbauda ievadītās informācijas korektumu. Ja visi dati ir derīgi, sistēma izveido lietotāja kontu un piešķir tam reģistrēta lietotāja tiesības. Ja kāds no laukiem aizpildīts nekorekti (piemēram, e-pasta adrese neatbilst formātam vai parole neatbilst noteiktajām prasībām), sistēma parāda kļūdas paziņojumu un neļauj pabeigt reģistrāciju, kamēr kļūdas nav novērstas."

Aprakstā ieteicams iekļaut gan tipiskus sistēmas izmantošanas scenārijus, gan nestandarta vai kļūdu situācijas, raksturojot lietotāja darbības un sistēmas reakciju katrā no tām. Aprakstā jānorāda:

- lietotāja darbību secība;
- kā sistēma reaģē uz katru darbību;

- kā izmantota sistēmas galvenā funkcionalitāte;
- kā sistēma rīkojas nestandarta vai kļūdu situācijās (piemēram, nekorekti ievaddati, nepietiekamas piekļuves tiesības, neeksistējoši dati, neveiksmīga autentifikācija u. c.).

Ja USE CASE diagrammā izmantotas «include» vai «extend» saites, tekstā jāpamato to izmantošana.

Piemēram:

"Lietošanas gadījums "Pievienot komentāru" izmanto «include» saiti ar lietošanas gadījumu "Autentificēties", jo komentārus drīkst pievienot tikai reģistrēti lietotāji."

"Lietošanas gadījums "Publicēt darbu" izmanto «extend» saiti ar lietošanas gadījumu "Nosūtīt administratoram apstiprināšanai", ja publicējamais saturs atbilst noteiktiem nosacījumiem."

2.2.5. Sistēmas koncepcijas izvērtējums

Šajā sadaļā jāizvērtē izstrādātā sistēmas koncepcija, izmantojot SVID (SWOT) analīzi. Analīzes mērķis ir novērtēt sistēmas stiprās un vājās puses, identificēt tās attīstības iespējas un iespējamos ārējos draudus, kā arī pamatot sistēmas turpmākās attīstības perspektīvas.

SVID analīzi ieteicams veidot tabulas veidā, atsevišķi raksturojot:

- stiprās puses (Strengths) – sistēmas priekšrocības, konkurētspējīgās īpašības un pozitīvie aspekti;
- vājās puses (Weaknesses) – sistēmas ierobežojumi, nepilnības vai funkcionalitāte, kas noslēguma darba ietvaros netiek realizēta;
- iespējas (Opportunities) – sistēmas turpmākās attīstības virzieni, papildu funkcionalitāte vai integrācijas iespējas nākotnē;
- draudi (Threats) – ārējie faktori, kas var ietekmēt sistēmas izmantošanu, attīstību vai konkurētspēju.

Katram SVID analīzes punktam jāsniedz īss pamatojums, nevis tikai atsevišķu atslēgvārdu uzskaitījums.

Pēc SVID analīzes jāformulē secinājumi, kuros jāizvērtē sistēmas kopējā perspektīva un jānorāda iespējamie pilnveidošanas virzieni. Ja sistēmas koncepcijā ir paredzēta papildfunkcionalitāte, kas noslēguma darba ietvaros netiek realizēta, to ieteicams aprakstīt tieši šajā sadaļā kā vienu no sistēmas turpmākās attīstības iespējām.

> SVID analīzei jābalstās uz iepriekš veiktās tirgus izpētes rezultātiem un izstrādāto sistēmas koncepciju. Analīzē nedrīkst iekļaut nepamatotus pieņēmumus vai funkcionalitāti, kas nav aprakstīta projekta dokumentācijā.

Piemērs:

Stiprās puses	Vājās puses
• Intuitīva lietotāja saskarne. • Ātra grāmatu meklēšana un filtrēšana.	• Nav paredzēta e-pasta paziņojumu nosūtīšana. • Sistēma neatbalsta datu eksportu.
Iespējas	**Draudi**
• Integrācija ar e-grāmatu platformām. • Automātiska ieteikumu sistēma, izmantojot MI.	• Tirgū jau pastāv līdzīgas sistēmas. • Autortiesību ierobežojumi publicētajiem darbiem.

2.2.6. Datu bāzes sistēmas projektēšana

1. Objektorientēts konceptuālais datu modelis

Šajā sadaļā jāizstrādā sistēmas objektorientētais konceptuālais datu modelis, kas raksturo sistēmas loģisko struktūru no objektorientētās programmēšanas viedokļa. Modelim jāparāda, kā sistēmā tiek organizēta un strukturēta informācija, kādi objekti (klases) veido sistēmu, kā tie savstarpēji saistīti un kādu funkcionalitāti tie nodrošina. Objektorientētajam modelim jāatbilst iepriekš aprakstītajai sistēmas koncepcijai, lietotāju lomām, USE CASE diagrammai un paredzētajai sistēmas funkcionalitātei (piemērus sk. 4. un 5.pielikumā).

Sadaļa sastāv no divām apakšsadaļām: **Klašu funkcijas un nozīme sistēmā** un **Klašu savstarpējās attiecības**.

✓ ***Klašu funkcijas un nozīme sistēmā***

Šajā apakšsadaļā jāiekļauj atsauce uz UML klašu diagrammu un jāsniedz katras diagrammā attēlotās klases apraksts.

Klašu diagrammai jāatbilst šādām prasībām.

- Diagrammā jābūt attēlotām vismaz piecām klasēm, kas veido sistēmas loģisko struktūru (neskaitot tehniskās vai palīgklases).
- Diagrammai jāatbilst sistēmas koncepcijai, lietotāju lomām, USE CASE diagrammai un paredzētajai sistēmas funkcionalitātei.
- Diagrammā jāiekļauj tikai tās klases, kurām ir nozīme sistēmas darbībā un kuras nepieciešamas sistēmas funkcionalitātes realizēšanai. Nav ieteicams iekļaut klases, kas nav pamatotas ar sistēmas koncepciju vai netiek izmantotas sistēmas darbībā.
- Katrai klasei jānorāda: klases nosaukums; atribūti ar datu tipiem un piekļuves līmeni (+, -, #); metodes ar piekļuves līmeni, nepieciešamajiem parametriem (ja tādi ir) un atgriežamo datu tipu.
- Starp klasēm jāizmanto tikai loģiski pamatotas attiecības. Nav ieteicams veidot savstarpējas attiecības starp klasēm tikai tādēļ, ka datu bāzē starp atbilstošajām tabulām eksistē ārējā atslēga. Klašu diagrammā attiecībām jāatspoguļo objektu sadarbība sistēmā.

- Pie katras attiecības jānorāda kardinalitāte.
- Diagrammai jābūt vizuāli pārskatāmai, ar loģisku elementu izvietojumu un minimālu līniju krustošanos.
- Diagrammā pareizi jāizmanto UML attiecību apzīmējumi: mantojuma bultiņai jābūt vērstai uz bāzes klasi, bet agregācijas vai kompozīcijas rombam jāatrodas veseluma klases pusē.

Klasēm jāatspoguļo sistēmas objekti un to uzvedība, nevis datu bāzes tabulas vai lietotāja saskarnes elementi (formas, pogas, logi u. c.). Klašu diagrammā nav ieteicams modelēt tehniskās vai palīgklases, kas nepiedalās sistēmas biznesa loģikas realizācijā.

Katras klases aprakstā jānorāda:

- klases nozīme sistēmā;
- kādu informāciju klase glabā;
- kādus atribūtus un metodes tā satur;
- klases funkcionalitāte, teksta veidā izskaidrojot, kādas darbības klase nodrošina sistēmā un kā tās tiek izmantotas;
- kāpēc šī klase ir nepieciešama un kā tā iekļaujas kopējā sistēmas darbības loģikā.

> Aprakstā jāparāda, kā konkrētā klase palīdz realizēt sistēmas funkcionalitāti un kā tā saistīta ar iepriekš aprakstītajām lietotāju vajadzībām, sistēmas koncepciju un USE CASE diagrammu. **Nav pieļaujama vienkārša klašu diagrammā norādīto metožu uzskaitīšana bez to nozīmes un pielietojuma apraksta.**

✓ ***Klašu savstarpējās attiecības***

Šajā apakšsadaļā jāizskaidro UML klašu diagrammā izmantotās attiecības starp klasēm un jāpamato izvēlētie objektorientētās modelēšanas risinājumi. Aprakstam jāpapildina klašu diagramma, paskaidrojot, kāpēc starp klasēm izveidotas konkrētās attiecības un kā tās atspoguļo sistēmas darbības loģiku. Nepietiek tikai uzskaitīt diagrammā attēlotās saites – katra būtiskā attiecība ir jāpamato.

Katram savstarpēji saistīto klašu pārim jānorāda:

- kāda veida attiecība izmantota (asociācija, agregācija, kompozīcija vai mantojums);
- kāpēc izvēlēts tieši šis attiecību veids;
- kā izvēlētā attiecība raksturo objektu savstarpējo saistību sistēmā;
- kā izvēlētā kardinalitāte raksturo objektu skaita attiecības.

Ja izmantota agregācija vai kompozīcija, jāpaskaidro, kā šī attiecība raksturo objektu dzīves ciklu.

Piemērs.

"Klase Nodaļa atrodas kompozīcijas attiecībās ar klasi Grāmata, jo nodaļa nevar pastāvēt bez konkrētas grāmatas. Dzēšot grāmatu, tiek dzēstas arī visas ar to saistītās nodaļas."

Ja diagrammā izmantots mantojums, papildus jāapraksta:

- kāpēc izvēlēta mantošanas attiecība;
- kādus atribūtus un metodes atvasinātā klase manto;
- kā mantojuma izmantošana samazina funkcionalitātes dublēšanos un vienkāršo sistēmas struktūru.

Piemērs.

"Klase Reģistrēts lietotājs manto klases Lietotājs atribūtus un metodes, jo abi lietotāju tipi izmanto kopīgu funkcionalitāti (piemēram, autentificēšanos un profila pārvaldību), bet reģistrētajam lietotājam ir pieejamas papildu iespējas, piemēram, satura publicēšana un komentāru pievienošana."

Ja klašu diagrammā izmantota mantošana, jāizskaidro, kāpēc izvēlēta tieši šī attiecība un kā tā palīdz izvairīties no atribūtu un funkcionalitātes dublēšanās.

> Aprakstā jāparāda, ka visas klašu savstarpējās attiecības ir loģiski pamatotas un atbilst iepriekš aprakstītajai sistēmas koncepcijai, lietotāju lomām un paredzētajai funkcionalitātei. Diagrammā nedrīkst būt nepamatotas vai neizmantotas attiecības.

2. Entītiju relāciju datu modelis

Šajā sadaļā jāapraksta sistēmas konceptuālais datu modelis, izmantojot entītiju relāciju (ER) modeli. Tajā jāparāda, kādas entītijas veido sistēmu, kādi dati tajās tiek glabāti un kā entītijas ir savstarpēji saistītas. ER modelis kalpo par pamatu loģiskā datu modeļa un datu bāzes tabulu struktūras izveidei.

ER diagrammu ieteicams ievietot pielikumā (paraugs pieejams 6. pielikumā), savukārt šajā sadaļā jāiekļauj atsauce uz attiecīgo pielikumu un jāsniedz diagrammas skaidrojums.

Sadaļas teksta daļā:

- jāapraksta katras entītijas funkcija sistēmā, kādi dati tajā tiek pārstāvēti, kāpēc šī entītija ir nepieciešama un kā tā piedalās sistēmas darbībā;
- katras relācijas aprakstā jāsniedz saistību skaidrojums divos teikumos (no abām pusēm). Piemēram: *Viens lietotājs var izveidot vairākus stāstus. Katrs stāsts pieder tikai vienam konkrētam lietotājam*;
- ja izmantoti EER elementi, jāskaidro to nepieciešamība un nozīme sistēmas darbībā.

ER datu modeļa apraksta piemērs ir dots 3.attēlā.

Aprakstā nav nepieciešams atkārtot visus diagrammā attēlotos atribūtus. Svarīgāk ir paskaidrot entītijas lomu sistēmā un tās nozīmi datu modelī.

3.2. Entītiju relāciju datu modelis

Datu bāzes projektēšanā datu kopu un saišu starp tām attēlošanai tika lietota realitāšu saišu diagramma, kas sastāv no divu veidu objektiem – entītēm un relācijām. ER diagramma (*sk.3.pielikumā*) sastāv no 10 entītijām, kas atspoguļo datu aprīti sistēmā.

- "**Lietotājs**" – uzskaita sistēmas lietotājus; tās atribūtu kopā ietilpst segvārds, e-pasta adrese, parole, loma, biogrāfija.
- "**Lietotāja stāsts**" – lietotāju rakstīto un vietnē publicēto stāstu saraksts; tās atribūtu kopā ietilpst nosaukums, apraksts, vecuma ierobežojums un darba statuss.
- .....

Datu bāzes relācijas parāda, kā divas vai vairākas entītijas ir savstarpēji saistītas.

- Starp lietotāju un lietotāja stāstu ir attiecība *viens pret daudziem*, jo lietotājs var uzrakstīt daudzus stāstus, bet viens konkrēts stāsts pieder tikai vienam konkrētam lietotājam.
- ...

3.att. Sistēmas ER datu modeļa apraksta piemērs

Prasības ER diagrammai

ER diagrammai jābūt izstrādātai **Pitera Čena (Peter Chen)** notācijā un jāatbilst šādām prasībām:

- jāattēlo visas entītijas, kas nepieciešamas sistēmas datu modeļa realizēšanai;
- katrai entītijai jānorāda visi būtiskie atribūti;
- kā identificējošie atribūti jānorāda priekšmeta jomā pastāvoši atribūti, kuru vērtības ļauj unikāli identificēt katru entītijas eksemplāru, piemēram, personas kods, ISBN, e-pasta adrese, reģistrācijas numurs vai cits unikāls kods. ER diagrammā nedrīkst iekļaut tehniskos identifikatorus, piemēram, ID, jo tie tiek ieviesti loģiskā datu modeļa izstrādes laikā, pārveidojot entītijas par datu bāzes tabulām;
- vienai entītijai var būt vairāki identificējošie atribūti, ja katrs no tiem atsevišķi spēj unikāli identificēt entītijas eksemplāru. Šādā gadījumā ER diagrammā jāatzīmē visi identificējošie atribūti;
- ārējās atslēgas (FK) ER diagrammā netiek attēlotas, jo tās pieder loģiskajam datu modelim;
- starp entītijām jānorāda visas relācijas, to nosaukumi un kardinalitātes;
- relāciju nosaukumiem jābūt loģiskiem un saprotamiem abos lasīšanas virzienos;
- entītiju nosaukumiem jābūt lietvārdiem, kas apzīmē reālus objektus vai jēdzienus, nevis darbības;
- entītijas jāveido tikai tiem objektiem, par kuriem sistēmā nepieciešams glabāt atsevišķus datus. Sistēmas lomas, statusi, kategorijas vai citi klasifikatori parasti tiek

modelēti kā atribūti, nevis atsevišķas entītijas, ja vien tiem nav savas neatkarīgas datu struktūras;

- daudz-pret-daudziem (M:N) relācijas konceptuālajā modelī drīkst attēlot tieši. To pārveidošana par starpentītijām tiek veikta loģiskā datu modeļa izstrādes laikā;
- izmantojot paplašināto ER modeli (EER), diagrammā jāattēlo arī specializācijas, mantošanas un vājo entītiju elementi;
- visi elementu nosaukumi un paskaidrojumi jāveido latviešu valodā;
- diagrammai jābūt vizuāli pārskatāmai, ar loģisku elementu izvietojumu un minimālu līniju krustošanos.

> ER diagrammā jāattēlo tikai sistēmā glabājamie dati un to savstarpējās attiecības. Tajā nedrīkst attēlot sistēmas procesus, funkcionalitāti vai lietotāju darbības, piemēram, reģistrāciju, autorizāciju, meklēšanu, filtrēšanu vai pieteikšanos.

3. Datu glabāšanas struktūra

Šajā sadaļā jāapraksta izstrādātās datu bāzes loģiskā struktūra, parādot, kā konceptuālais datu modelis ir pārveidots par relāciju datu bāzes tabulām. Jāizskaidro, kā dati tiek organizēti tabulās, kā savstarpēji saistītas tabulas un kā tiek nodrošināta datu glabāšana relāciju datu bāzē.

Katras tabulas aprakstā jānorāda (piemēru sk. 7.pielikumā):

- tabulas nosaukums un īss apraksts par tās nozīmi sistēmā, paskaidrojot, kāda informācija tabulā tiek glabāta un kā tā tiek izmantota sistēmas darbībā;
- visi tabulas lauki, norādot katra lauka nosaukumu un datu tipu. Simbolu virkņu laukiem (VARCHAR, CHAR u. c.) papildus jānorāda maksimālais atļautais garums;
- papildinformācija par laukiem, norādot, kuri lauki ir primārās (PK) vai ārējās (FK) atslēgas, kā arī būtiskākos datu integritātes ierobežojumus un formāta prasības (piemēram, NOT NULL, UNIQUE, DEFAULT, CHECK).

Datu glabāšanas struktūra jāattēlo datu bāzes tabulu saišu shēmas veidā (paraugs ir pieejams 8.pielikumā). Shēmu ieteicams ievietot pielikumā, savukārt šajā sadaļā jāiekļauj atsauce uz attiecīgo pielikumu un tās skaidrojums. Tabulu saišu shēmai jāatbilst šādām prasībām:

- jāattēlo visas datu bāzes tabulas, kas nepieciešamas sistēmas darbībai;
- tabulu struktūrai jāatbilst iepriekš izstrādātajai ER diagrammai;
- katrā tabulā jānorāda tabulas nosaukums, visi lauki, datu tips katram laukam, simbolu virkņu laukiem (VARCHAR, CHAR u.c.) jānorāda arī maksimālais garums;
- primārās atslēgas jāapzīmē ar PK, bet ārējās atslēgas – ar FK;

- starp tabulām jāattēlo visas relācijas, izmantojot izvēlētā diagrammu veidošanas rīka grafiskos apzīmējumus;
- relācijām jāatbilst iepriekš izstrādātajam konceptuālajam datu modelim;
- relāciju datu modelī nedrīkst būt tiešas daudzi pret daudziem (M:N) relācijas;
- ja konceptuālajā modelī izmantota 1:1 relācija, tabulu struktūrā jānodrošina tās korekta realizācija (piemēram, izmantojot ārējo atslēgu ar UNIQUE ierobežojumu);
- diagrammai jābūt vizuāli pārskatāmai, ar loģisku tabulu izvietojumu un minimālu līniju krustošanos.

Tabulu struktūrai loģiski jāizriet no iepriekš izstrādātās ER diagrammas. Projektēšanas gaitā ir pieļaujamas izmaiņas, kas nepieciešamas relāciju datu modeļa realizācijai (piemēram, tehnisko primāro atslēgu, ārējo atslēgu vai starptabulu ieviešana).

4. Datu bāzes atbilstības izvērtējums trešajai normālformai

Šajā sadaļā jāizvērtē izstrādātās datu bāzes struktūras atbilstība trešās normālformas (3NF) prasībām. Analīzei jābūt balstītai uz konkrētiem piemēriem no izstrādātās datu bāzes, nevis tikai teorētisku normālformu definīciju izklāstu.

Sadaļas mērķis ir pamatot, ka izveidotā datu bāzes struktūra nodrošina efektīvu datu glabāšanu, samazina datu dublēšanos un palīdz uzturēt datu integritāti.

Sadaļā obligāti jāiekļauj:

- datu bāzes struktūras izvērtējums, pamatojot tās atbilstību vai neatbilstību trešās normālformas (3NF) prasībām;
- konkrēti piemēri no izstrādātās datu bāzes (tabulas, lauki un relācijas), kas pamato veikto izvērtējumu;
- skaidrojums, kā projektēšanas laikā novērsta datu dublēšanās un nodrošināta datu savstarpējā loģiskā saistība;
- pamatojums gadījumiem, ja kāda tabula pilnībā neatbilst 3NF prasībām. Jāpaskaidro, kāpēc pieņemts šāds projektēšanas risinājums un kā tas ietekmē sistēmas darbību;
- ieteikumi datu bāzes struktūras turpmākai pilnveidošanai, ja tādi ir nepieciešami.

Izvērtējumā ieteicams atsaukties uz iepriekš izstrādāto ER diagrammu un tabulu saišu shēmu, paskaidrojot, kā konceptuālais datu modelis projektēšanas gaitā pārveidots par normalizētu relāciju datu bāzes struktūru.

Aprakstā nav ieteicams tikai pārstāstīt pirmās, otrās un trešās normālformas teorētiskās definīcijas. Vērtējums tiks piešķirts par spēju analizēt un pamatot tieši savas izstrādātās datu bāzes struktūru.

Piemērs

"Ja tabulā Grāmata kopā ar autora kodu atkārtoti tiek glabāts arī autora vārds, uzvārds un e-pasta adrese, šie lauki funkcionāli ir atkarīgi no autora, nevis tieši no grāmatas primārās atslēgas. Lai nodrošinātu atbilstību 3NF, informācija par autoru jāizdala atsevišķā tabulā Autors, bet tabulā Grāmata jāglabā ārējā atslēga uz autoru."

Ja kāda tabula apzināti neatbilst 3NF, jāsniedz pamatojums šādam risinājumam (piemēram, projekta apjoma ierobežojumu vai sistēmas vienkāršošanas dēļ).

2.2.7. Datu bāzes sistēmas realizācija un testēšana

Šajā nodaļā jāapraksta datu bāzes sistēmas praktiskā realizācija un tās darbības pārbaude. Balstoties uz iepriekš izstrādāto datu modeli, jāparāda, kā izveidota datu bāzes struktūra, kā nodrošināta datu integritāte un kā realizētas sistēmas funkcionālās prasības.

Sadaļa sastāv no trim apakšsadaļām.

1. **Datu bāzes realizācija**

Šajā apakšsadaļā jāapraksta datu bāzes struktūras praktiskā realizācija, pamatojoties uz iepriekš izstrādāto tabulu saišu shēmu (paraugs ir pieejams 9.pielikumā).

Jāiekļauj:

- datu bāzes izveides SQL skripts (*SQL skriptam pilnībā jāatbilst dokumentācijā aprakstītajai tabulu struktūrai*);
- primāro un ārējo atslēgu realizācija;
- datu integritātes ierobežojumu izmantošana (NOT NULL, UNIQUE, CHECK, DEFAULT u. c.);
- ārējo atslēgu darbības nosacījumi, tostarp ON UPDATE un ON DELETE, ja tie izmantoti;
- izmantoto ierobežojumu izvēles pamatojums;
- papildu datu bāzes mehānismi, piemēram, indeksi, trigeri vai skati, ja tie ir ieviesti.
- SQL skriptam jāatbilst tabulu saišu shēmai un jābūt izpildāmam bez sintakses kļūdām.

> Ja datu korektuma nodrošināšanai tiek izmantoti nosacījumi, kurus nevar pilnībā attēlot tabulu saišu shēmā vai SQL skriptā, tie īsi jāpaskaidro teksta daļā.
>
> Ja ārējām atslēgām tiek izmantoti ON UPDATE vai ON DELETE nosacījumi, dokumentācijā jāpamato katras izvēlētās darbības atbilstība sistēmas darbības loģikai. Nepietiek tikai norādīt izmantoto SQL konstrukciju.

2. Datu pārvaldības darbību realizācija un integritātes pārbaude

Šajā apakšsadaļā jāparāda datu pievienošanas, labošanas un dzēšanas darbību realizācija:

- datu pievienošana (INSERT);
- datu labošana (UPDATE);
- datu dzēšana (DELETE).

Datu pārvaldības darbībām jābūt saistītām ar konkrētām sistēmas vajadzībām un jāievēro datu bāzē definētie integritātes ierobežojumi.

Papildus jāveic integritātes nosacījumu pārbaude, izmantojot gan korektus, gan apzināti nekorektus testa datus. Piemēram, var pārbaudīt:

- vai iespējams ievietot ierakstu, ja obligātais lauks nav aizpildīts;
- vai iespējams ievietot dublējošu vērtību laukā ar UNIQUE ierobežojumu;
- vai iespējams ievietot ārējās atslēgas vērtību, kurai nav atbilstoša ieraksta saistītajā tabulā;
- vai darbojas CHECK ierobežojumi;
- kā sistēma reaģē, dzēšot vai mainot saistītos ierakstus.

Katram pārbaudes piemēram jānorāda izmantotā SQL komanda, sagaidāmais rezultāts un testa rezultāts.

3. Funkcionālo prasību realizācija un testēšana

Šajā apakšsadaļā jāapraksta sistēmas funkcionālo prasību realizācija un to testēšana, lai pārliecinātos, ka izstrādātie SQL risinājumi nodrošina paredzēto datu izgūšanu un apstrādi atbilstoši sistēmas koncepcijai.

Šajā sadaļā jāiekļauj visas funkcionālās prasības, kas aprakstītas Uzdevuma nostādnes apakšsadaļā "Funkcionalitāte datu izgūšanai un apstrādei". Katrai funkcionālajai prasībai jāparāda tās realizācija, izmantotais SQL risinājums un testēšanas rezultāti.

Katras funkcionālās prasības aprakstā ieteicams iekļaut:

- funkcionālās prasības aprakstu;
- izmantoto SQL vaicājumu vai citu SQL risinājumu;
- SQL risinājuma darbības skaidrojumu;
- testēšanas pārskatu.

Funkcionālās prasības aprakstā jānorāda prasības mērķis, tās nozīme sistēmā un lietotāja veicamās darbības. SQL risinājuma aprakstā jāpaskaidro izmantotās SQL konstrukcijas un to nozīme prasības realizācijā.

Pēc katras funkcionālās prasības realizācijas jāveic tās testēšana. Testēšanas pārskatu ieteicams noformēt tabulas veidā, norādot testa numuru, ievaddatus, sagaidāmo rezultātu un testa rezultātu.

Katras funkcionālās prasības testēšanai ieteicams izveidot vismaz trīs testa gadījumus:

- pozitīvo testu, kur ievaddati pilnībā atbilst noteiktajiem nosacījumiem;
- robežgadījumu, kur tiek pārbaudītas minimālās vai maksimālās pieļaujamās vērtības;
- negatīvo testu, kur ievaddati neatbilst noteiktajiem nosacījumiem vai vaicājums nedrīkst atgriezt nevienu ierakstu.

Testa ievaddatiem jābūt savstarpēji atšķirīgiem un jāaptver dažādas sistēmas darbības situācijas.

Sagaidāmajā rezultātā ieteicams norādīt konkrētus ierakstus, kuriem jābūt atlasītiem, vai skaidri norādīt, ka atlasītajiem kritērijiem neatbilst neviens ieraksts. Testa rezultātā pietiek norādīt, vai tests ir veiksmīgs vai neveiksmīgs. Ja tests ir neveiksmīgs, papildus jāapraksta konstatētā problēma un nepieciešamie uzlabojumi.

Funkcionālo prasību realizācijas un testēšanas paraugs ir pieejams 10.pielikumā.

2.2.8. Nobeigums

Nobeigumā jāapkopo noslēguma darba rezultāti, jāizvērtē darba mērķa un uzdevumu izpilde, kā arī jānorāda datu bāzes sistēmas prototipa turpmākās attīstības iespējas. Ieteicams īsi apkopot paveikto darbu, formulēt galvenos secinājumus un raksturot sistēmas pilnveidošanas virzienus, neveidojot tos kā atsevišķas apakšsadaļas.

Kopsavilkumā īsi jāapraksta paveiktais darbs, uzsverot būtiskākos rezultātus. Jānorāda, kāds datu bāzes sistēmas prototips ir izstrādāts, kādi galvenie projektēšanas un realizācijas posmi ir veikti, kā arī kādas sistēmas funkcionālās prasības ir realizētas un pārbaudītas.

Secinājumos jāizvērtē, vai noslēguma darba mērķis ir sasniegts un vai izvirzītie uzdevumi ir izpildīti. Jānovērtē, vai izstrādātais datu bāzes sistēmas prototips nodrošina plānoto datu glabāšanu, pārvaldību un apstrādi un vai tas var kalpot par pamatu turpmākai tīmekļa vietnes vai citas informācijas sistēmas izstrādei. Ja darba gaitā kāda sākotnēji plānotā funkcionalitāte netika realizēta vai tika mainīta, jānorāda šādu izmaiņu iemesli un to ietekme uz izstrādāto prototipu.

Noslēgumā ieteicams norādīt arī datu bāzes sistēmas prototipa turpmākās attīstības iespējas. Jāapraksta funkcionalitāte, kas netika realizēta noslēguma darba ietvaros, bet kuru būtu lietderīgi ieviest, turpinot sistēmas izstrādi. Piemēram, var paredzēt lietotāja saskarnes vai citas klientprogrammas izstrādi, automātisku e-pasta paziņojumu nosūtīšanu, datu eksportu dažādos formātos, ārējo sistēmu integrāciju, papildu lietotāju lomas, statistikas pārskatus vai citas funkcijas, kas paplašinātu sistēmas iespējas.

2.2.9. Informācijas avoti

Šajā sadaļā jāuzskaita visi informācijas resursi, kas izmantoti noslēguma darba izstrādes gaitā – grāmatas, zinātniskie un populārzinātniskie raksti, mācību materiāli, interneta resursi, intervijas, audio un videomateriāli, kā arī citi avoti, uz kuriem ir atsauces darba tekstā.

2.2.10. Pielikumi

Pielikumos pievieno materiālus, kas ir pārāk detalizēti vai apjomīgi, lai tos iekļautu pamattekstā, bet kas nepieciešami, lai pilnībā izprastu un novērtētu izstrādāto darbu. Tie var būt dažādas diagrammas (UML klašu diagrammas, ER diagrammas, lietošanas gadījumu diagrammas u. c.), datu bāzes shēmas, tabulu vai datu izdruku ekrānuzņēmumi (screenshot), testēšanas scenāriji un rezultāti.

PIELIKUMI

1. pielikums

Titullapas paraugs

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

Moduļa “Datu bāzu tehnoloģijas”

NOSLĒGUMA DARBS

Tēma: Internetveikala datu uzskaites un apstrādes sistēmas prototipa izstrāde

Rīga

2026./2027. m.g.

2. pielikums

Satura rādītājs

SATURS

IEVADS ... **3**

1. UZDEVUMA NOSTĀDNE ... **4**

1.1. Sistēmas funkcionalitāte ... 5

1.1.1. Vispārējā sistēmas funkcionalitāte ... 5

1.1.2. Datu pārvaldības funkcionalitāte ... *5*

1.1.3. Funkcionalitāte datu izgūšanai un apstrādei ... 6

1.2. Sistēmas lietotāji un to mijiedarbība ar sistēmu ... 7

1.2.1. Sistēmas lietotāji ... 7

1.2.2. Lietotāju uzvedība un mijiedarbība ar sistēmu ... 8

2. SISTĒMAS KONCEPCIJAS IZVĒRTĒJUMS ... **10**

3. DATU BĀZES SISTĒMAS PROJEKTĒŠANA ... **12**

3.1. Objektorientēts konceptuālais datu modelis ... 12

3.1.1. Klašu funkcijas un nozīme sistēmā ... 12

3.1.2. Klašu savstarpējās attiecības ... 14

3.2. Entītiju relāciju datu modelis ... 16

3.3. Datu glabāšanas struktūra ... 18

3.4. Datu bāzes atbilstības izvērtējums trešajai normālformai ... 22

4. DATU BĀZES SISTĒMAS REALIZĀCIJA UN TESTĒŠANA ... **24**

4.1. Datu bāzes realizācija ... 24

4.2. Datu pārvaldības darbību realizācija un integritātes pārbaude ... 28

4.3. Funkcionālo prasību realizācija un testēšana ... 32

NOBEIGUMS ... **40**

INFORMĀCIJAS AVOTI ... **41**

PIELIKUMI ... **42**

1.pielikums. Lietošanas gadījumu diagramma ... 42

2.pielikums. Klašu diagramma ... 43

3.pielikums. ER diagramma ... 44

4.pielikums. Datu bāzes tabulu shēma ... 45

Sistēmas lietotāju apraksta piemērs

1.2.1. Sistēmas lietotāji

Izstrādātajā sistēmā ir definētas trīs lietotāju lomas – **viesis**, **reģistrēts lietotājs** un **administrators**. Katrai lomai ir pieejama atšķirīga funkcionalitāte un piekļuves tiesības.

Viesis var apskatīt publiski pieejamo informāciju, meklēt grāmatas un iepazīties ar to aprakstiem, taču nevar pievienot komentārus, vērtējumus vai publicēt saturu. Lai izmantotu pilnu sistēmas funkcionalitāti, viesim jāreģistrējas un jāizveido lietotāja konts.

Reģistrēts lietotājs var meklēt un lasīt grāmatas, pievienot vērtējumus un komentārus, publicēt savus darbus, kā arī pārvaldīt savu profilu. Lai izmantotu funkcijas, kas saistītas ar datu pievienošanu vai rediģēšanu, lietotājam jābūt autentificētam.

Administrators pārvalda sistēmas saturu un lietotājus. Viņš var rediģēt vai dzēst jebkura lietotāja publicēto informāciju, pārvaldīt lietotāju kontus, pārskatīt sistēmā iesniegto saturu, apstiprināt vai noraidīt publicēšanai iesniegtos darbus, kā arī nepieciešamības gadījumā bloķēt lietotāju kontus.

Sistēmas lietotāju lomu mijiedarbība ar sistēmu ir attēlota 1. pielikumā pievienotajā lietošanas gadījumu (USE CASE) diagrammā, kurā parādītas lietotāju lomas, to lietošanas gadījumi un savstarpējās atkarības (include un extend).

Objektorientētā konceptuālā datu modeļa apraksta piemērs

3. DATU BĀZES SISTĒMAS PROJEKTĒŠANA

Šajā nodaļā aprakstīta datu bāzes sistēmas projektēšana un izstrāde. Sistēmas projektēšana tiek veikta vairākos secīgos posmos, pakāpeniski pārejot no konceptuāla sistēmas apraksta līdz datu bāzes struktūrai, kas ir gatava realizācijai relāciju datu bāzes pārvaldības sistēmā.

Vispirms tiek izstrādāts objektorientētais konceptuālais datu modelis UML klašu diagrammas veidā, kas raksturo sistēmas galvenos objektus, to īpašības, funkcionalitāti un savstarpējās attiecības.

Pēc tam tiek izveidots konceptuālais entītiju relāciju datu modelis ER diagrammas veidā, kurā aprakstītas datu entītijas, to atribūti un savstarpējās relācijas.

Balstoties uz ER diagrammu, tiek izstrādāts loģiskais datu modelis — datu bāzes tabulu saišu shēma, kurā attēlotas tabulas, primārās un ārējās atslēgas, kā arī citi datu integritātes nodrošināšanai nepieciešamie ierobežojumi.

Noslēgumā tiek veikta datu bāzes normalizācijas analīze, izvērtējot tabulu atbilstību trešajai normālformai (3NF) un pamatojot pieņemtos projektēšanas risinājumus.

3.1. Objektorientēts konceptuālais datu modelis

Sistēmā informācija tiek organizēta, izmantojot objektorientētu pieeju, kur katra klase raksturo konkrētu sistēmas objektu ar tam raksturīgajiem datiem un darbībām. Objektorientētais modelis ļauj attēlot sistēmas loģisko struktūru, objektu savstarpējo sadarbību un galvenās funkcijas vēl pirms datu bāzes projektēšanas.

Objektorientētais konceptuālais datu modelis ir attēlots **2. pielikumā** UML klašu diagrammas veidā. Diagrammā redzamas sistēmas galvenās klases, to atribūti, metodes un savstarpējās attiecības.

Sistēmu veido divpadsmit savstarpēji saistītas klases, kas nodrošina lietotāju, lietotāju stāstu, klasisko grāmatu, nodaļu, žanru, grāmatzīmju, vērtējumu un jautājumu pārvaldību.

3.1.1. Klašu funkcijas un nozīme sistēmā

Lietotāji – ir sistēmas centrālā klase, kas glabā informāciju par sistēmas lietotājiem un nodrošina autentifikācijas, profila pārvaldības un konta administrēšanas funkcionalitāti. Ar šo klasi ir saistīta lielākā daļa pārējo sistēmas objektu, jo gandrīz visas darbības sistēmā tiek veiktas konkrēta lietotāja vārdā.

.....

3.1.2. Klašu savstarpējās attiecības

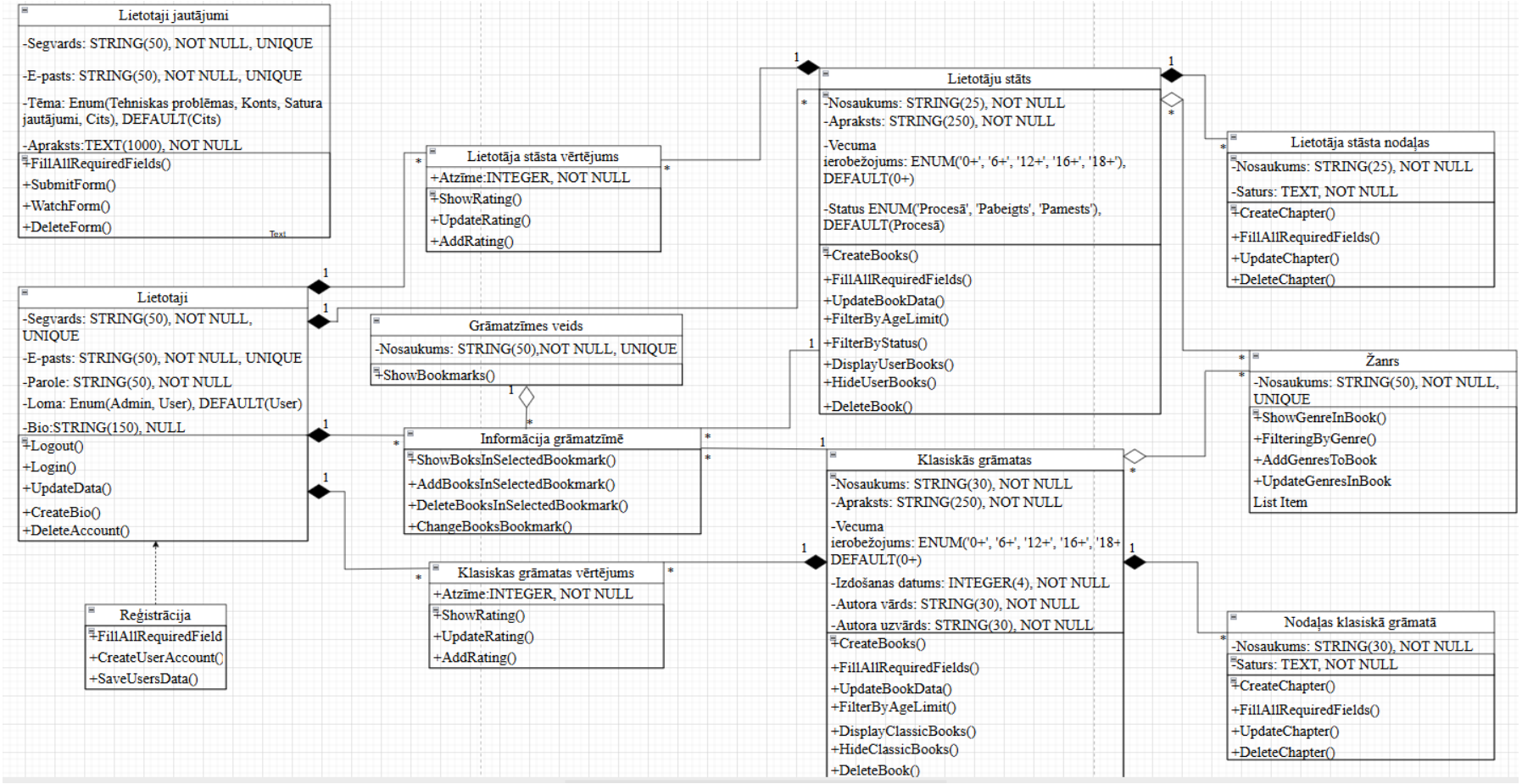
Klašu diagrammā izmantotas asociācijas, agregācijas un kompozīcijas, kas raksturo sistēmas objektu savstarpējās attiecības un to dzīves ciklu.

Klase **Lietotāji** ir sistēmas centrālais objekts un ir saistīta ar vairākām citām klasēm. Vienam lietotājam var būt vairāki izveidoti stāsti, vairākas grāmatzīmes, vairāki piešķirti vērtējumi un vairāki iesniegti jautājumi. Šīs attiecības diagrammā modelētas kā kompozīcijas, jo attiecīgie dati ir tieši saistīti ar konkrēto lietotāju un bez tā zaudē savu nozīmi.

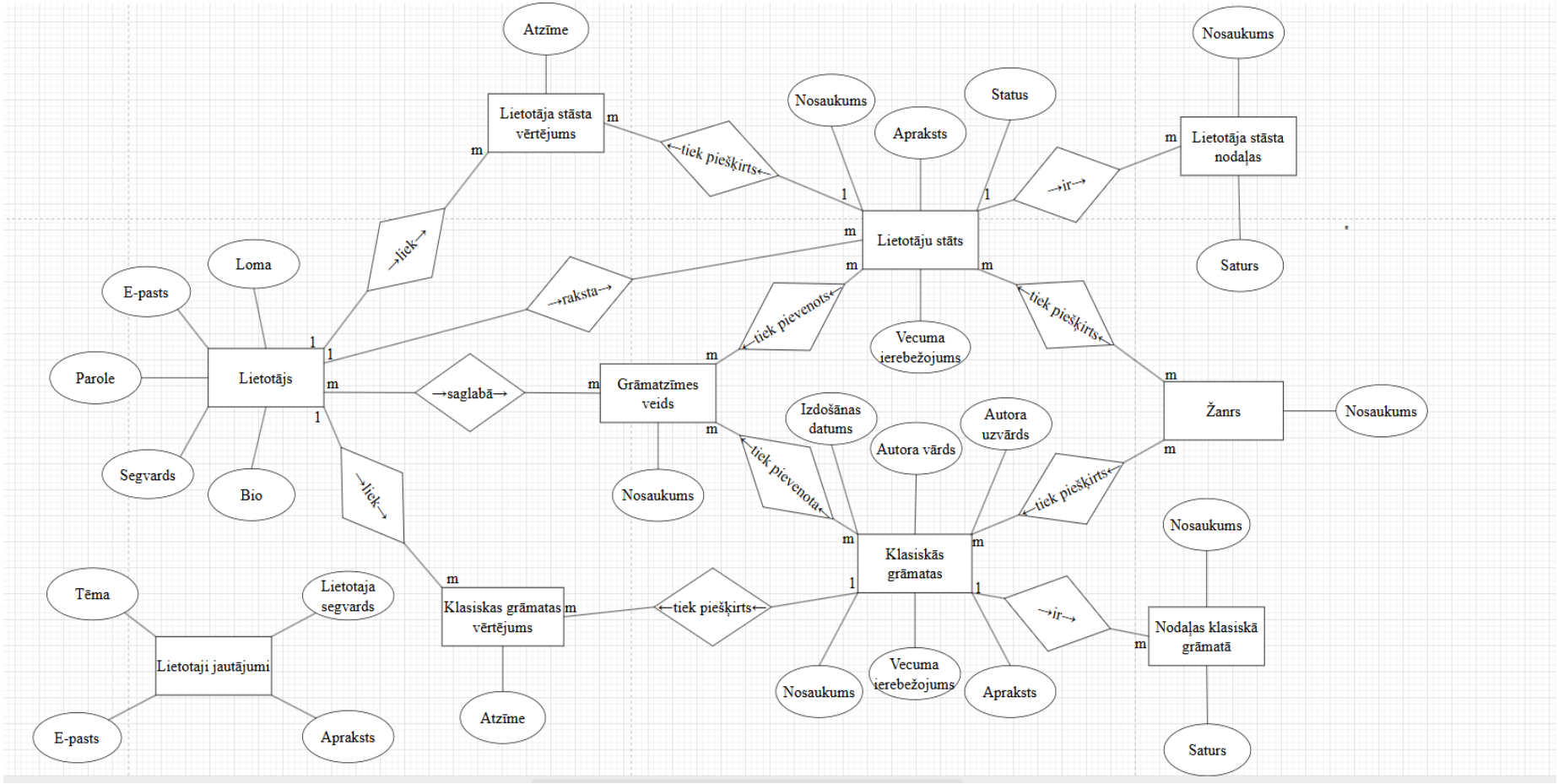
....

Klašu diagrammā nav izmantota mantošana. Tomēr sistēmas struktūru nākotnē būtu iespējams vienkāršot, ieviešot abstraktu Grāmata klasi, no kuras mantotu gan Lietotāja stāsts, gan Klasiskās grāmatas, jo abas klases satur vairākus kopīgus atribūtus un nodrošina līdzīgu funkcionalitāti. Tāpat būtu iespējams ieviest kopīgu Vērtējums bāzes klasi, no kuras mantotu Klasiskās grāmatas vērtējums un Lietotāja stāsta vērtējums, tādējādi samazinot funkcionalitātes dublēšanos un padarot sistēmas objektorientēto modeli pārskatāmāku.

Klašu diagrammas piemērs



ER diagrammas piemērs



Datu glabāšanas struktūras apraksta piemērs

3.3. Datu glabāšanas struktūra

Datu bāze sastāv no 13 tabulām, kas satur informāciju par lietotājiem, klasiskajām grāmatām, lietotāju stāstiem, lietotāju vērtējumiem, grāmatzīmēm un žanriem. Datubāzes projektēšanas procesā, lai tabulas atbilstu realizējamajām prasībām un tiktu novērsta datu dublēšanās, tiek veikta to optimizēšana jeb normalizācija. 4. pielikumā ir parādīta datu bāzes struktūra, kas demonstrē kolonu izvietojumu pa tabulām, kā arī tabulu savstarpējo saistību.

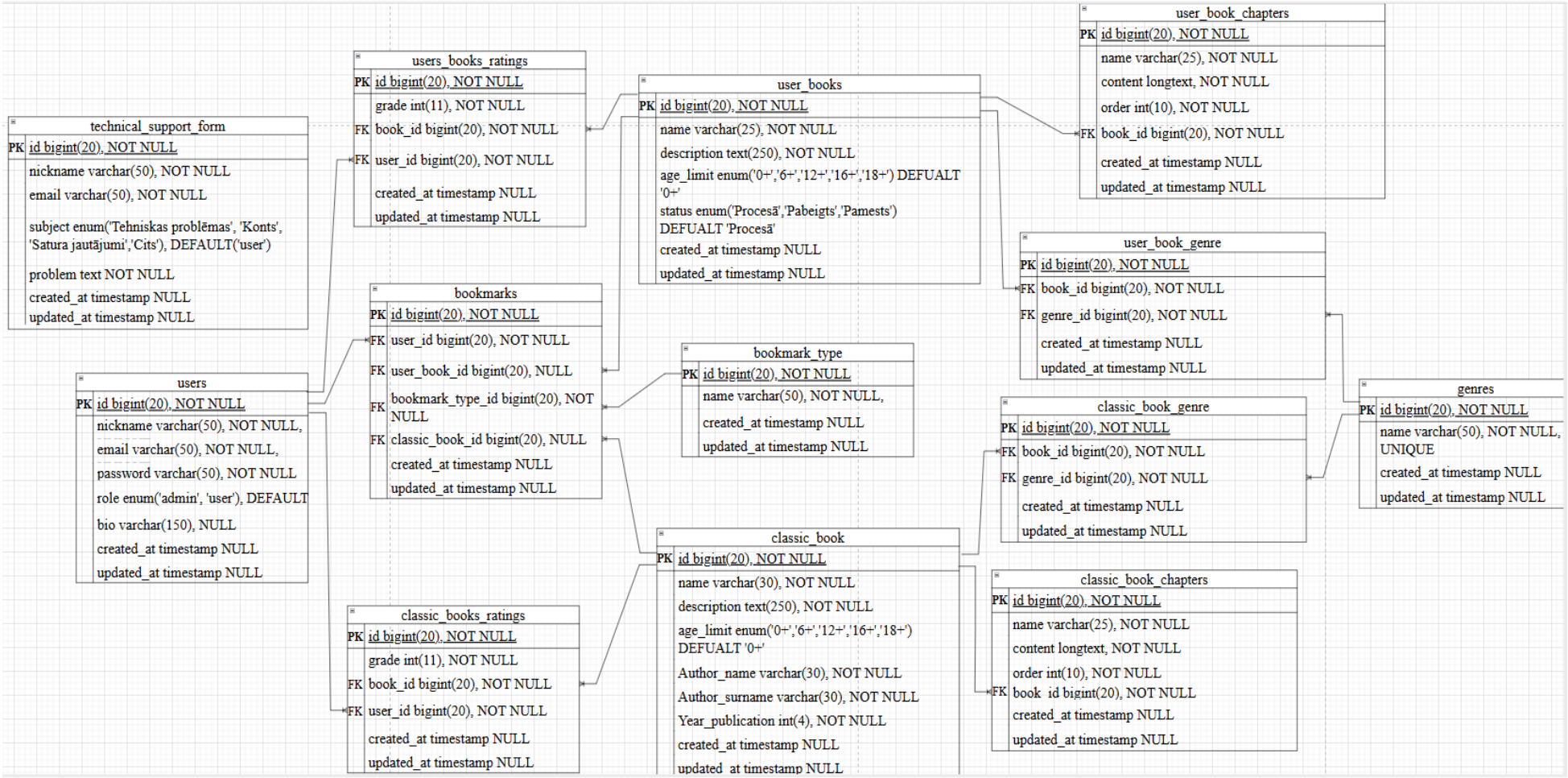
Tabulā "users" tiek glabāta informācija par sistēmā reģistrētajiem lietotājiem. Tajā ir unikāls lietotāja ID, e-pasta adrese, segvārds, parole, lietotāja loma sistēmā (pēc noklusējuma ir parastais lietotājs), papildu lauks īsai biogrāfijai un ieraksta izveides un atjaunināšanas laiks.

3.1. tabula

Tabulas **„users”** struktūra

Nr.	Nosaukums	Tips	Garums	Piezīme
1	id	bigint		lietotāja kods, primārā atslēga
2	email	varchar	50	lietotāja epasta adrese, formāts test@test.com
3	nickname	varchar	50	reģistrētā lietotāja segvārds
4	password	varchar	50	reģistrētā lietotāja šifrēta parole
5	role	enum('admin', 'user')		Reģistrēta lietotāja loma. Pēc noklusējuma šis ir parasts lietotājs bez privilēģijām.
6	….			

Datu bāzes tabulu saišu shēmas piemērs



Datu bāzes realizācijas apraksta piemērs

4. DATU BĀZES SISTĒMAS REALIZĀCIJA UN TESTĒŠANA

4.1. Datu bāzes realizācija

Datu bāzes realizācija tika veikta, izmantojot SQL CREATE TABLE komandas. Katra tabula tika izveidota atbilstoši iepriekš izstrādātajai tabulu saišu shēmai, nodrošinot datu integritāti ar primārajām un ārējām atslēgām, kā arī datu integritātes ierobežojumiem.

Tabulas **bookmarks** realizācija

Tabula bookmarks paredzēta lietotāju izveidoto grāmatzīmju glabāšanai. Tā nodrošina saikni starp lietotāju, grāmatzīmes veidu un objektu, kuru lietotājs vēlas saglabāt (lietotāja stāstu vai klasisko grāmatu).

Tabulas izveidei izmantota šāda SQL komanda:

```
CREATE TABLE bookmarks (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,

    user_id BIGINT NOT NULL,
    bookmark_type_id BIGINT NOT NULL,
    user_book_id BIGINT NULL,
    classic_book_id BIGINT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,

    CONSTRAINT fk_bookmarks_user
        FOREIGN KEY (user_id) REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE,

    CONSTRAINT fk_bookmarks_type
        FOREIGN KEY (bookmark_type_id) REFERENCES bookmark_type(id) ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT fk_bookmarks_user_book
        FOREIGN KEY (user_book_id) REFERENCES user_books(id) ON UPDATE CASCADE ON DELETE
SET NULL,

    CONSTRAINT fk_bookmarks_classic_book
        FOREIGN KEY (classic_book_id) REFERENCES classic_book(id) ON UPDATE CASCADE ON
DELETE SET NULL
);
```

Izmantoto ierobežojumu pamatojums

Ierobežojums	Pamatojums
PRIMARY KEY (id)	Katrai grāmatzīmei tiek piešķirts unikāls identifikators
NOT NULL (user_id, bookmark_type_id)	Grāmatzīmi nav iespējams izveidot bez lietotāja un grāmatzīmes veida norādīšanas
FOREIGN KEY (user_id)	Nodrošina, ka grāmatzīmi var izveidot tikai eksistējošs lietotājs
ON DELETE CASCADE (user_id)	Dzēšot lietotāju, automātiski tiek dzēstas arī visas viņa grāmatzīmes, jo bez lietotāja tās zaudē savu nozīmi
ON UPDATE CASCADE	Ja tiek mainīta primārās atslēgas vērtība saistītajā tabulā, izmaiņas automātiski tiek atjauninātas arī ārējās atslēgas laukā. Tas novērš nekorektu saišu veidošanos
ON DELETE RESTRICT (bookmark_type_id)	Neļauj dzēst grāmatzīmes veidu, kamēr tas tiek izmantots vismaz vienā grāmatzīmē.
ON DELETE SET NULL (user_book_id)	Dzēšot lietotāja stāstu, grāmatzīme tiek saglabāta, bet saite uz dzēsto stāstu tiek automātiski dzēsta (iestatīta uz NULL)

Papildu paskaidrojums

Ārējo atslēgu darbības (ON UPDATE un ON DELETE) izvēlētas atbilstoši sistēmas loģikai. Piemēram, lietotāja dzēšanas gadījumā grāmatzīmes zaudē savu nozīmi, tāpēc izmantots ON DELETE CASCADE. Savukārt, ja tiek dzēsta grāmata vai lietotāja stāsts, grāmatzīmes ierakstu var saglabāt, noņemot tikai atsauci uz dzēsto objektu (ON DELETE SET NULL).

Funkcionālo prasību realizācijas un testēšanas piemērs

Funkcionālā prasība – Grāmatu meklēšana pēc vairākiem kritērijiem (Advanced Filter)

Funkcionālās prasības apraksts

Šī funkcionalitāte nodrošina iespēju lietotājam atlasīt grāmatas pēc vairākiem kritērijiem vienlaikus. Lietotājs var norādīt autoru, izdošanas gada intervālu un minimālo lietotāju vērtējumu. Meklēšanas rezultātos tiek attēlotas tikai tās grāmatas, kas vienlaikus atbilst visiem izvēlētajiem nosacījumiem.

SQL vaicājums

….

SQL risinājuma apraksts

Vaicājumā tiek izmantota tabulu classic_book un classic_books_ratings apvienošana (JOIN), lai iegūtu informāciju par grāmatām un to vērtējumiem. Ieraksti tiek filtrēti pēc autora un izdošanas gada (WHERE), pēc tam grāmatas tiek grupētas (GROUP BY) un aprēķināts katras grāmatas vidējais vērtējums (AVG). Nosacījums HAVING atlasa tikai tās grāmatas, kuru vidējais vērtējums nav mazāks par četrām ballēm. Rezultāti tiek sakārtoti alfabētiskā secībā pēc grāmatas nosaukuma.

Testēšanas pārskats

Testa Nr.	**Ievaddati**			**Sagaidāmais rezultāts**	**Testa rezultāts**
	Autors	***Izdošanas gads***	***Vērtējums***		
1	Arturs Konans Doils	1925–1928	≥ 4	Tiek atlasītas grāmatas "Pazudušā pasaule" un "Miglas zeme".	Veiksmīgs
2	Arturs Konans Doils	1900–1910	≥ 5	Netiek atlasīts neviens ieraksts.	Veiksmīgs
3	Jānis Bērziņš	2000–2025	≥ 3	Tiek atlasīta grāmata "Programmēšanas pamati".	Veiksmīgs
4	Viljams Šekspīrs	1500–1600	≥ 5	Tiek atlasīta grāmata "Hamlets".	Veiksmīgs
5	Neeksistējošs autors	2000–2025	≥ 3	Netiek atlasīts neviens ieraksts.	Veiksmīgs

Secinājums

Testēšanas rezultāti apliecina, ka izstrādātais SQL vaicājums korekti realizē funkcionālo prasību. Meklēšana tiek veikta pēc vairākiem kritērijiem vienlaikus, izmantojot tabulu apvienošanu (JOIN), filtrēšanu (WHERE), grupēšanu (GROUP BY), agregātfunkciju (AVG) un grupēto rezultātu filtrēšanu (HAVING). Visos testa gadījumos iegūtais rezultāts atbilda sagaidāmajam rezultātam.